

ADVANCED URL SHORTENER WEB APPLICATION

A Project Report

Submitted in partial fulfillment of the requirements
for the Advanced Web Development Assignment

Submitted by

Krishna Kishore

Institute

Innomatics Research Labs

Submission Date

18 January 2026

ABSTRACT

The Advanced URL Shortener Web Application is a web-based system developed using Flask, SQLAlchemy, and SQLite. The purpose of this project is to provide a simple and secure way to shorten long URLs and allow users to manage their shortened links through authentication.

The application supports user registration and login, URL shortening, redirection to original URLs, and storage of URLs on a per-user basis. This project focuses on backend development concepts such as authentication, database relationships, and request handling, along with basic frontend design using HTML and CSS.

Contents

ABSTRACT	2
1. INTRODUCTION	4
2. PROBLEM STATEMENT	4
3. OBJECTIVES OF THE PROJECT	4
4. TECHNOLOGIES USED	5
5. SYSTEM ARCHITECTURE	5
6. DATABASE DESIGN	6
7. APPLICATION WORKFLOW	6
8. CHALLENGES FACED	10
9. CONCLUSION	10
10. FUTURE ENHANCEMENTS	11

1. INTRODUCTION

In the modern digital environment, sharing web links has become a routine activity. Many websites generate long and complex URLs, which are difficult to remember and inconvenient to share. This often creates problems when sending links through messages, emails, or social media platforms.

A URL shortener helps solve this problem by converting long URLs into shorter and more manageable links. These shortened links redirect users to the original website when accessed. This project implements an advanced URL shortener that also includes user authentication and personalized URL management.

2. PROBLEM STATEMENT

The key issues addressed by this project are:

- Long URLs are inconvenient to share and manage
- Basic URL shorteners do not provide user-specific access
- Users cannot track or revisit their previously shortened URLs
- Lack of authentication can lead to data misuse

The project aims to overcome these challenges by integrating secure login functionality and database-backed URL storage.

3. OBJECTIVES OF THE PROJECT

The objectives of this project are:

- To shorten long URLs into unique short codes
- To implement user signup and login functionality
- To allow users to store and view their shortened URLs
- To ensure URLs are accessible only to their respective users
- To provide proper redirection from short URLs to original URLs

4. TECHNOLOGIES USED

The following technologies were used in the development of this project:

Backend Technologies

- **Python** – Core programming language used for backend logic
- **Flask** – Lightweight web framework used to handle routing and HTTP requests
- **Flask-Login** – Used for implementing user authentication and session management

Database Technologies

- **SQLAlchemy ORM** – Used for interacting with the database using Python objects
- **SQLite** – Lightweight relational database used for storing users and URLs

Frontend Technologies

- **HTML** – Used to structure the web pages
- **CSS** – Used for styling and layout of the application
- **Bootstrap** – Used for basic responsive design and improved user interface components

Development & Tools

- **Visual Studio Code** – Code editor used for development
- **Python Virtual Environment (venv)** – Used to manage project dependencies

5. SYSTEM ARCHITECTURE

The application follows a modular architecture. Authentication logic and URL handling logic are separated into different route files. Database models are defined independently to maintain clarity and organization.

The Flask application factory pattern is used to initialize the application and database. Templates and static files are stored in separate directories, making the project easier to understand and maintain.

6. DATABASE DESIGN

The database consists of two main tables:

User Table

- Stores user credentials
- Each user has a unique username
- Passwords are stored in encrypted (hashed) format

ShortURL Table

- Stores original URLs and their shortened codes
- Each shortened URL is linked to a user
- Supports user-specific URL storage

There is a one-to-many relationship between users and URLs, where one user can have multiple shortened URLs.

7. APPLICATION WORKFLOW

1. The user opens the home page
2. New users register using a username and password
3. Existing users log in using their credentials
4. After login, the user is redirected to the dashboard
5. The user enters a long URL and clicks the shorten button
6. A unique short code is generated and saved
7. The short URL redirects to the original URL
8. Users can view only their own shortened URLs
9. The user can log out securely

The application begins with a home page that provides navigation options for users to either log in or sign up.

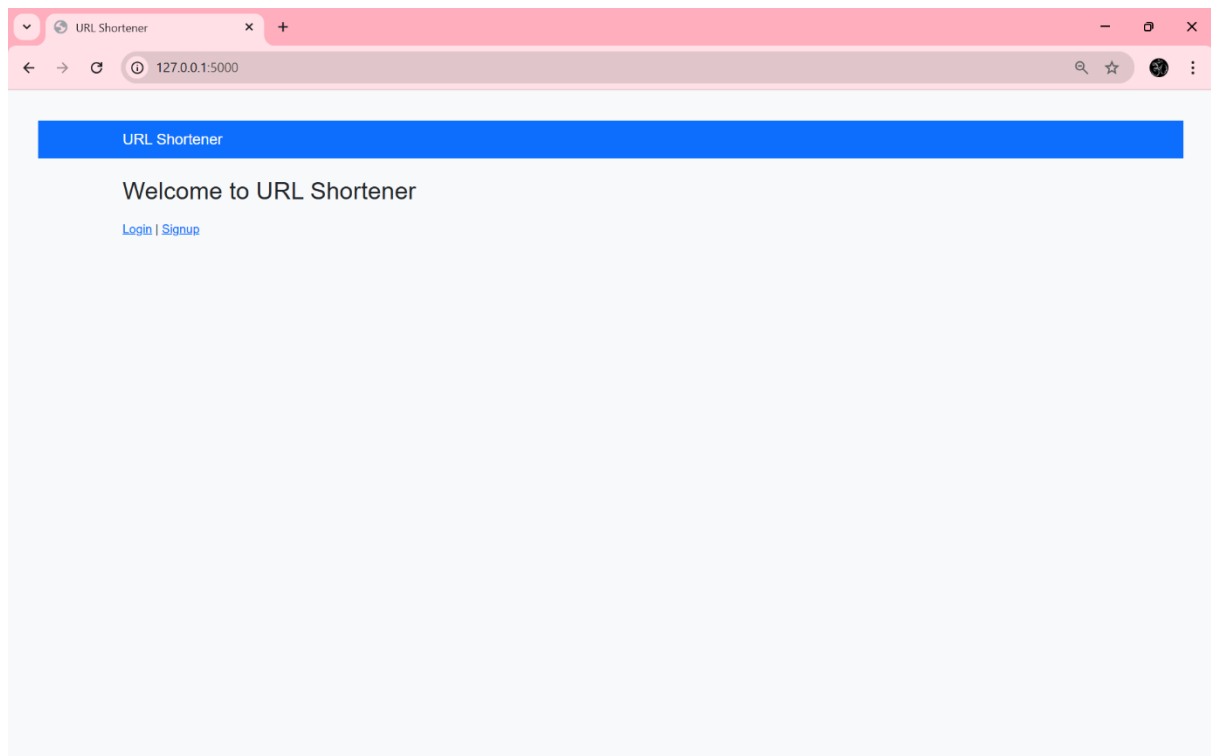


Figure 1: Home Page of the URL Shortener Application

New users can create an account by entering a unique username and password on the signup page.

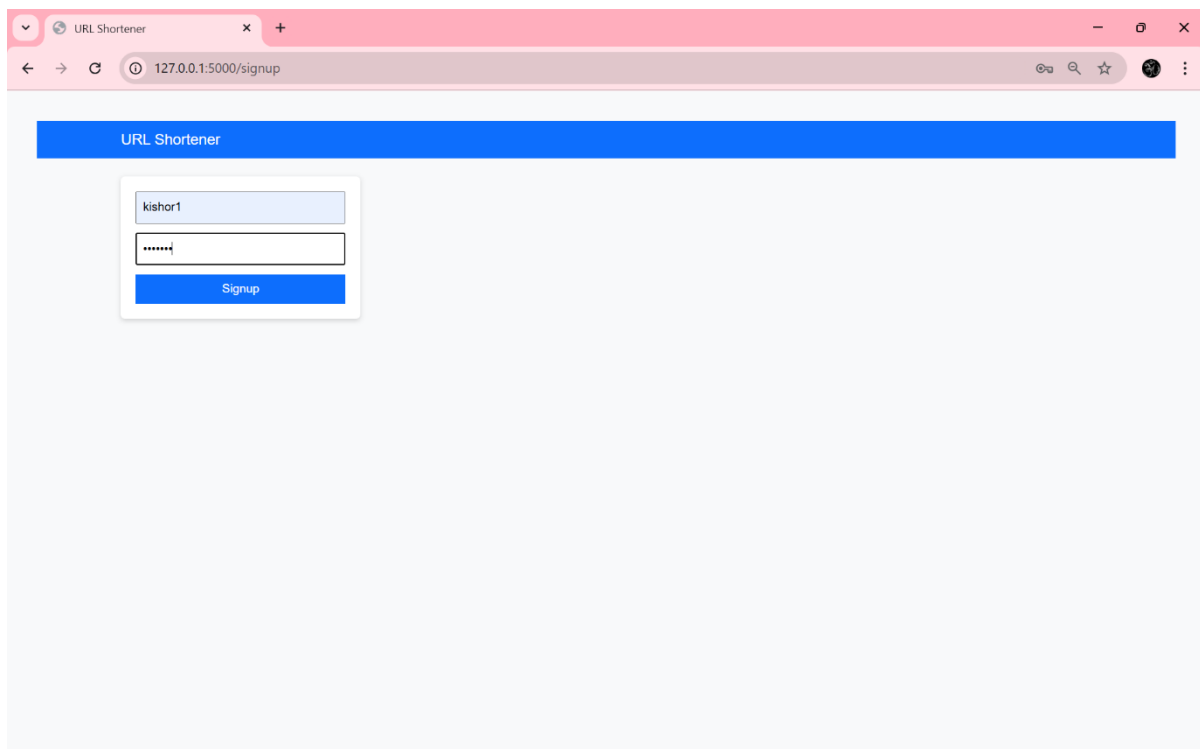


Figure 2: User Signup Page

After registration, users log in using their credentials to access the application dashboard.

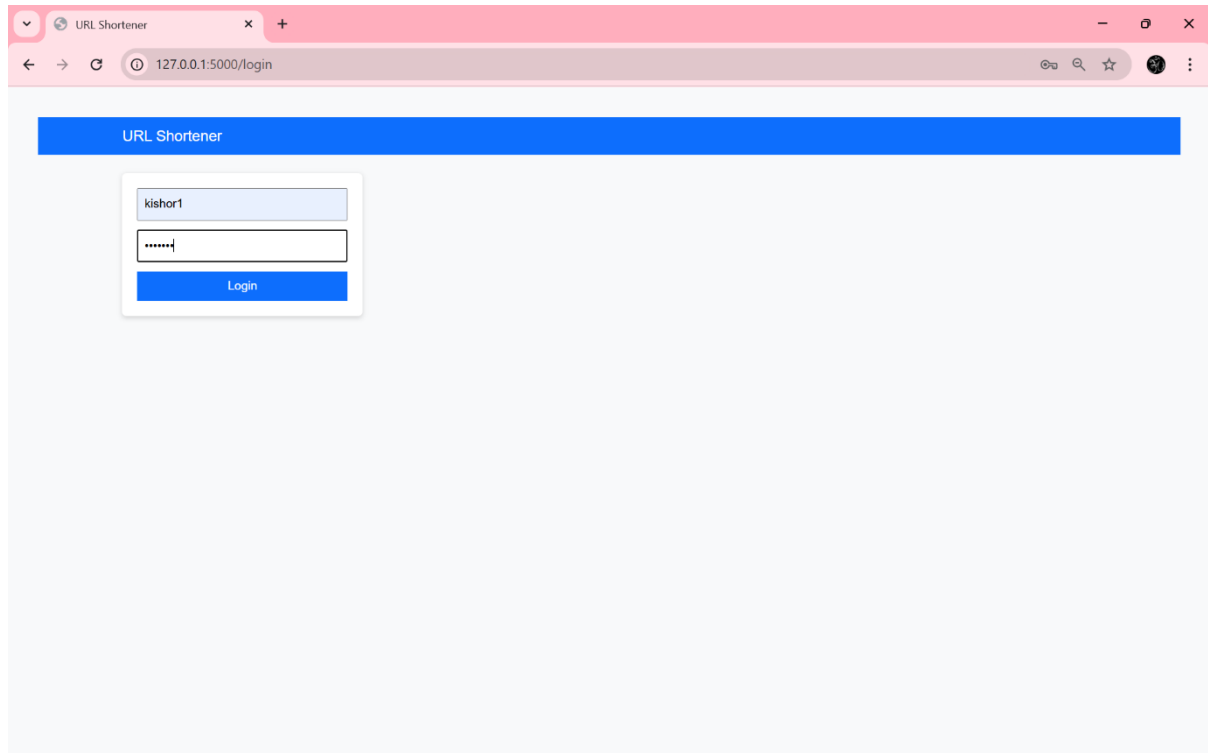


Figure 3: User Login Page

Once logged in successfully, the user is redirected to the dashboard where URL shortening functionality is available.

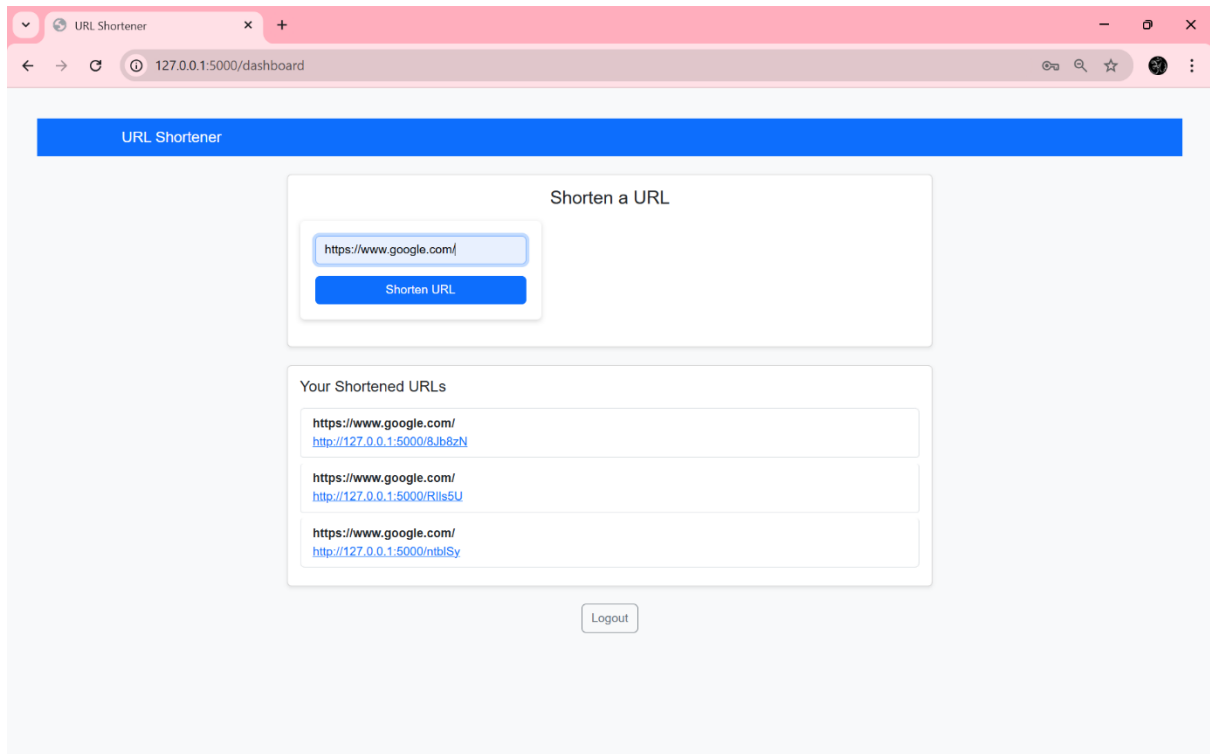


Figure 4: User Dashboard After Login

The user can enter a long URL in the input field and generate a shortened URL, which is then displayed on the dashboard.

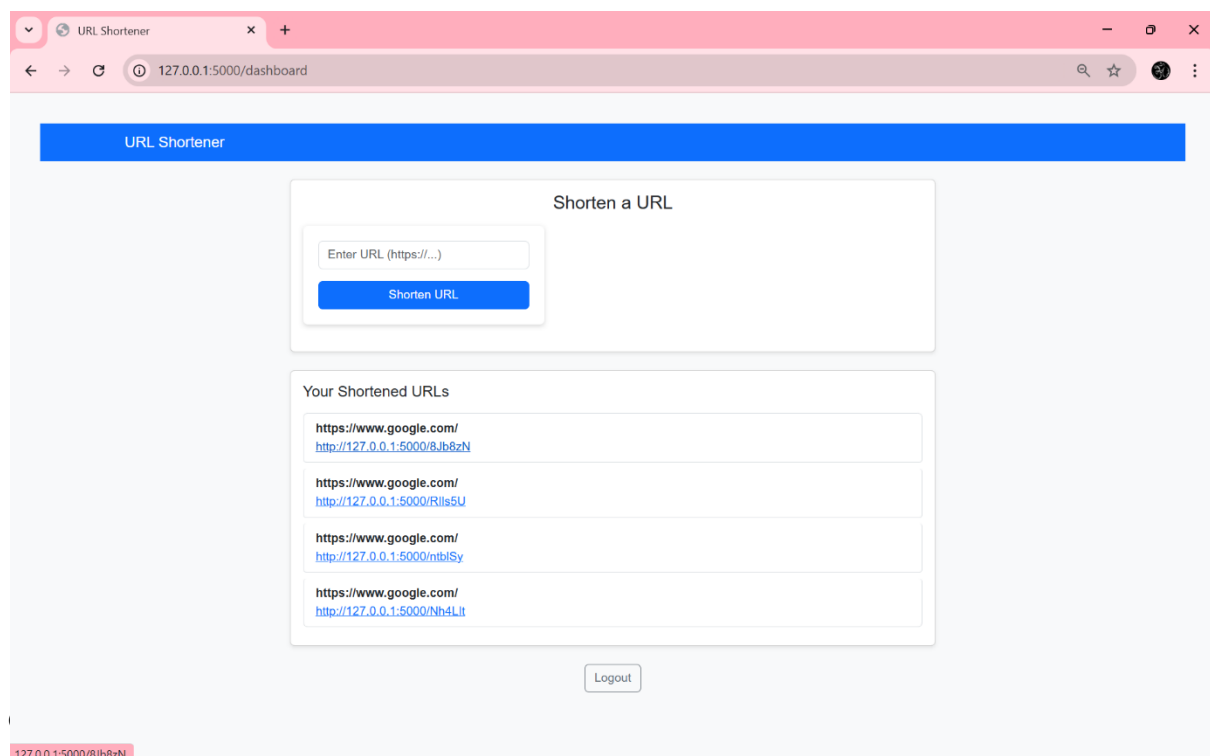


Figure 5: User Dashboard After Login

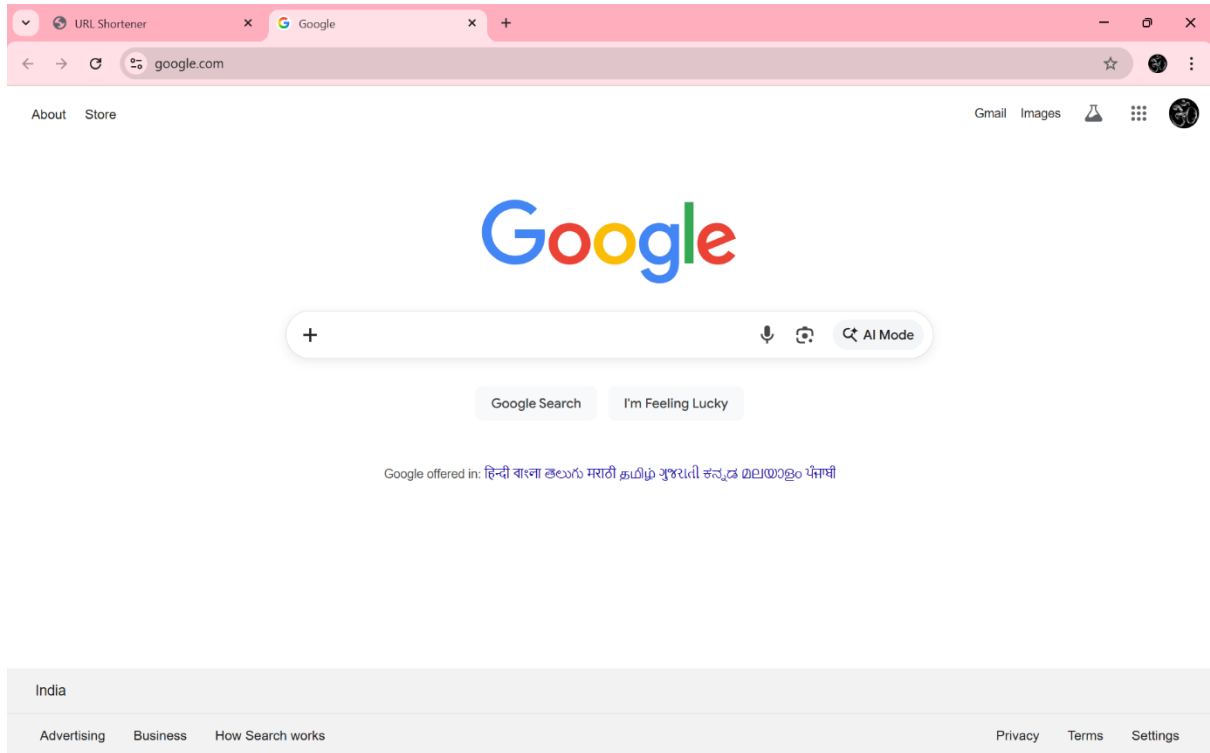


Figure 6: Redirection to Original URL Using Shortened Link

8. CHALLENGES FACED

Some challenges encountered during the development of this project include:

- Implementing secure user authentication
- Designing correct database relationships
- Handling form validation and error cases
- Debugging model and field mismatches in SQLAlchemy

These challenges were resolved through testing, debugging, and a clear understanding of Flask and database concepts.

9. CONCLUSION

The Advanced URL Shortener Web Application successfully fulfills the project requirements. It provides a secure and efficient way to shorten URLs while allowing users to manage their data individually.

Through this project, practical experience was gained in Flask development, authentication handling, database design, and backend application structure.

10. FUTURE ENHANCEMENTS

Possible future improvements include:

- Adding analytics such as click count tracking
- Enhancing the user interface using Bootstrap
- Allowing users to create custom short codes
- Deploying the application to a cloud platform